Human Upper-Body Motion Capturing using Kinect

Wei-Chia Kao and Shih-Chung Hsu
Department of Electrical Engineering
National Tsing-Hua University
Hsin-Chu, Taiwan
e-mail: chvjohnnf@gmail.com

Chung-Lin Huang
Department of Applied Informatics and Multimedia
Asia University
Tai-Chung, Taiwan
e-mail: clhuang@asia.edu.tw

***Abstract*—This paper proposes a real-time upper human motion capturing method to estimate the positions of upper limb joints by using Kinect. For human articulated motion capturing, the body part self-occlusion is a nontrivial problem. The system consists of hybrid action type recognition, body part segmentation, and offset compensation. The hybrid action type classifier consists of Adaboost and Random Forest classifier. The major contributions of this paper are offset compensation and self-occluded joint recovery. The offset is the difference between the output and the ground truth. The offset compensation is proposed by correcting the estimated locations of the joints. For different user action type, we train an appropriate offset classifier for offset compensation. Finally, we propose a post-processing to justify the effectiveness of the offset compensation.**

Keywords—Motion Capturing, Action Type Recognition, Body Part Segementation, Adaboost, Random Forest.

I. INTRODUCTION

This paper introduces a real time system to track the human motion and estimate the motion parameters. A robust human motion capturing system has many applications, such as human-computer interface (HCI), secure monitor, and medical treatment. Microsoft [1] proposes the motion capture by launching the first consumer depth camera, Kinect. The Kinect-based motion capturing system becomes more popular because of its lower cost compared with the marker-based or sensor-based devices.

Recently, many different motion capture approaches have been developed [6, 7], of which vision-based technologies have widely been proposed [7]. The availability of real time depth image capturing technologies have been developed [2, 9~13]. The vision-based methods can be divided into two categories: model-based methods [2, 4, 9] and example-based methods [5, 11~13]. The former relies on human body model and object tracking technology, and the latter collects different body motions and uses object recognition technology.

Model-based approaches estimate the human joint parameters based on the matching between the input images and human model. Ganapathi *et al.* [2] proposed a probabilistic filtering frame-work and employ a highly accurate generative model. They combined discriminative part detections with local hill-climbing and the smooth likelihood function. Schwarz *et al*. [9] present a method for human full-body pose estimation by detecting the anatomical landmarks in the 3D data and fitting with a skeleton body model. Baak *et al.* [4] combined local optimization with global retrieval techniques to efficiently extract pose features from the depth data. The major disadvantage of model-based approach is the complexity of human model generation which is time consuming. In the model-based tracking [2], the re-initialization and self-correction for tracking error are not easy.

Example-based approaches search the database to find the pose images that is most similar to the current image as an output. This method requires a considerable large image database. Wang *et al.* [5] proposed two different kinds of classifiers, bottom-up joint position classifier and top-down skeleton classifier, which are combined to achieve the final results. Shotton *et al.* [1] proposed a method to locate the 3D positions of body joints by using the random forest classifier. However, the offset exists between the estimated motion parameter and the ground truth. To obtain the offset of the body part joint, [3] develop the offset classifier by using the Random Forest. In [1], to avoid the problem of model retrieval in the large model database, they applied tree searching which can be accelerated by the GPU [20].

Different from [1, 15], we develop a two-stage classifier to identify the human action types and then segment the human silhouette into different body parts for joint localization. Different from[3], we develop a better offset classifier to improve the accuracy of estimated motion parameters. Our system can recover the self-occluded body part, and then justify the offset compensation of the joint locations.

II. PRE-PROCESSING AND MODEL TRAINING

We develop a segmentation method to divide each human depth map into 16 different body parts (as shown in Fig. 1) as the training data for model training. First, to extract the foreground human object, we use flood-fill algorithm [26] to label the largest connected component in the depth map which is extracted as the foreground human silhouette. Based on the depth and 2-D location of the pixels of human silhouette, we may apply the seeded region growing technique [27] to segment the human silhouette. Second, we differentiate the occluded region from the occluding region (*e.g.*, face occluded by palm) based on the depth discontinuity. Third, we manually select the centroid of each occluding or occluded body part region in the silhouette image. By using the centroids as the seeds, we apply the region growing to partition the human silhouette into 16 body part regions. Finally, we may collect the training pose samples from different people.

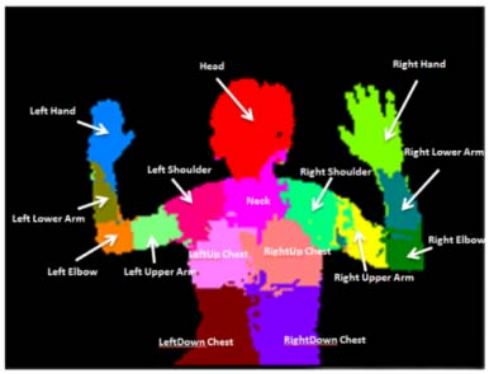


Fig. 1. Human upper body segmented into 16 body parts.

III. Hybrid Action Classifier

We propose a hybrid action classifier to identify the four action types: two hands down, left hand up, right hand up, and two hands up. The hybrid classifier consists of Adaboost and Random Forest classifier. Random Forest classifier compares two blocks as local feature to identify the action type. Adaboost classifier takes advantage of global feature. If the outcome of Random Forest classifier is left hand up or right hand up, we apply AdaBoost for further verification. The hybrid action classifier generates more accurate results.

Random Forest classifier takes advantage of the local features, whereas the AdaBoost classifier uses the global feature. The latter is more effective than the former to identify these two specific action types "right hand up" and "left hand up". On the other hand, once "two hands up" or "two hands down" is identified, we consider that the two hands in front of the chest which can be effectively identified by comparing local characteristics.

AdaBoost recognizes the right/left hand action type more precisely by using the global feature, whereas Random Forest classifier relies on the local features. If the two results of the two classifiers are different, we choose the identified action type in the preceding frame. To improve the accuracy, we take advantage of the temporal dependency and apply the correction using the recognized action types in five consecutive frames.

IV. Body Part Classifier and Joint Recovery

For each action type, there exists a corresponding body part classifier which is a random forest classifier. To collect the training samples, we record the patches around pixel $\boldsymbol{x}$ of a particular depth image $\boldsymbol{I}$. Here, we use the displacement $\{\boldsymbol{u}, \mathbf{v}\}$ as the split function parameter. Spilt function f_θ is defined as

$$f_\theta = d_I\left(\boldsymbol{x} + \frac{\boldsymbol{u}}{d_I(\boldsymbol{x})}\right) - d_I\left(\boldsymbol{x} + \frac{\boldsymbol{v}}{d_I(\boldsymbol{x})}\right) \quad (1)$$

where $d_I(\boldsymbol{x})$ is the depth at pixel $\boldsymbol{x}$ and parameter $\theta=\{\boldsymbol{u},\boldsymbol{v}\}$. We normalize the displacement by using $1/d_I(\boldsymbol{x})$ to ensure that the features are depth invariant. After training, we have the probability distribution from the leaf node, $P_t(c|I,\boldsymbol{x})$, where t indicates the t^{th} tree, and c is the label of body part. For each input pixel $\boldsymbol{x}$, its body part category is determined as

$$\text{Output Body type } c^* = max_c\, P_t(c|I,\boldsymbol{x}) \quad (2)$$

A. Action Type Correction

Incorrect action type leads to inappropriate body part classifier. Temporal dependency and spatial variance can be used to identify the correctness of the action type. If the action type is incorrect, then the segmented body parts will not be accurate. The accuracy of the segmented body parts will be evaluated by the homogeneity measure. Based on several consecutive identified action types, we may identify the correctness of current action type. For each action type, we may compute the variance of each body part to determine the homogeneity of the label pixels in each body part. If the total number of body part of which the variance is too large, this action type is incorrect.

B. Joint Recovery

Based on the spatial-temporal relationship of human action, we may solve the self-occlusion problem of the body parts which causes joint disappearance. The body part joints are labeled as shown in Fig. 2. The occluded body part joint recovery is based on spatial/temporal correlation between the joints. Here, we use current body part joint to infer the other body part joint, and start examining the body part joints from the head joint label "0".

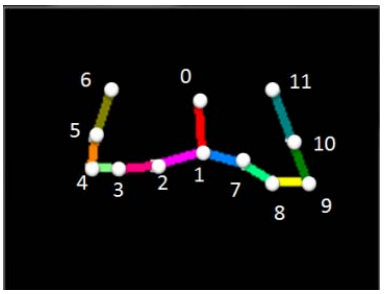


Fig. 2. The digit numbers are used to label the body part joints

There are three scenarios for body part joint recovery. In the 1st case, we consider the body part joints labeled 0, 1, 2, and 7. If the estimated body part joint is ambiguous, it is replaced by the results of the previous frame. In the 2nd case, we consider the body part joints labeled 3, 5, 8, and 10. These joints are located between two joints. It is possible to infer by the previous joint and the following joint. For example, joint 2 is the previous joint of joint 3, and joint 4 is the succeeding joint of joint 3. We can use $P_4 - P_2$ vector to refer joint 3. If we did not detect the previous joint or the succeeding joint of joint 3, we cannot compute the vector referring to joint 3. So, we use the estimated location of the same joint detected in previous frame for the current joint.

In the 3rd case, we consider body part joints 4, 6, 9, and 11. These joints are located at the end of the arm. They can be recovered based on the spatial relationship of the preceding and succeeding joints. For example, Joint 5 is the preceding joint of Joint 6, and Joint 4 is the preceding joint of Joint 5. So we can use $P_5 - P_4$ to generate 3-D vectors to estimate Joint 6. If the previous joint is not detected or the joint before the previous joint is not detected, then we directly estimate the joint in the previous frame using current joint. Here, we assign a constant weight factor W for recovering the joints. The weight factor is

defined as the distance between two joints P_0 and P_1 as the de-normalization factor.

Denotations: P_i^t is the location of the i^{th} joint, i indicates one of the twelve joints and t is the time. N indicates the joint disappearance, $W=\|(P_0 - P_1)\|$. For disappearing joint P_i^t, we may refer to other joints based on the following three cases.

Case 1: For $i \in \{0, 1, 2, 7\}$, $P_i^{t-1} \rightarrow P_i^t$.

Case 2: For $i \in \{3, 5, 8, 10\}$

(a) If $P_{i+1}^t \neq N$ and $P_{i-1}^t \neq N$,

then $W \cdot \frac{(P_{i+1}^t - P_{i-1}^t)}{\sqrt{|(P_{i+1}^t - P_{i-1}^t)|}} + P_{i-1}^t \rightarrow P_i^t$.

(b) If $P_{i+1}^t = N$ or $P_{i-1}^t = N$,

then $P_i^{t-1} \rightarrow P_i^t$.

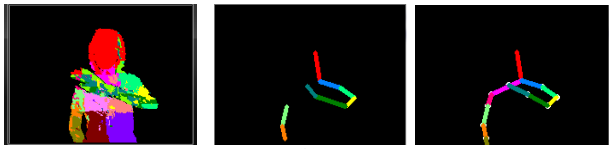
Case 3: For $i \in \{4, 6, 9, 11\}$

(a) If $P_{i-2}^t \neq N$ and $P_{i-1}^t \neq N$,

then $W \cdot \frac{(P_{i-1}^t - P_{i-2}^t)}{\sqrt{|(P_{i-1}^t - P_{i-2}^t)|}} + P_{i-1}^t \rightarrow P_i^t$.

(b) If $P_{i-2}^t = N$ or $P_{i-1}^t = N$,

then $P_i^{t-1} \rightarrow P_i^t$.



(a) (b) (c)

Fig. 3. (a) Segmented body part with self-occlusion, (b) the body part joints disappear by self-occlusion, and (c) after body part joint recovery.

V. OFFSET COMPENSATION

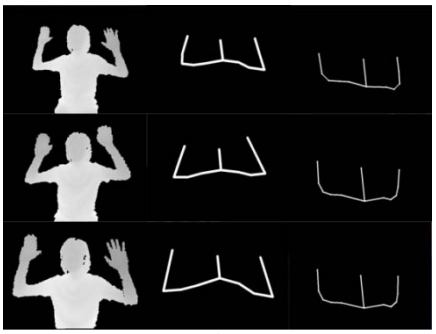
We propose the offset compensation to add the offsets to the estimated body part joints. The offsets are the difference between the estimated joints of the body part and the ground truth. Given the body part joints, (*i.e.*, the center of each body part), we use the offset classifier to find the offset for compensating the estimated body part joints. Finally, we propose a method to justify whether the compensated body part joints are correct or not based-on the matching scores.

We only use twelve of the sixteen joints without torso. The offset is a 12×3 dimension vector which is used to compensate the estimated body part joints. The offset classifier training by using the random forest has several advantages: (1) decision tree classifier with less depth. (2) No retraining for the offset classifiers for an additional action type. (3) The offsets in the same leaf node are more consistent. To train the offset classifier, we collect the offset training set for different action types.

Before training, we need to normalize the skeleton and minimize the variance of the offset training data of different persons. We define the head joint P_0 as the frame centroid and then translate the j^{th} joint vector as $D_j = P_j - P_0$, and add the weighted normalized D_j to ${P_0}^{new}$ as

$$ {P_j}^{new} = {P_0}^{new} + \frac{D_j}{\|D_j\|_2} * W \tag{3} $$

for j=1,…12, where W is the weight. The normalization results for the same pose from different persons are illustrated in Fig. 4.



(a) (b) (c)

Fig. 4. Normalization: (a) the depth silhouette image, (b) the original skeleton, and (c) the normalized skeleton.

The offset training data of the every action needs to be normalized to avoid producing different offset F_j for different persons. The offset is the difference between each joint P_j and the correct joint ${P_j}^{GT}$. If the offset is too large, then it will be ignored. The normalization is defined as

$$ F_j = \frac{P_j - {P_j}^{GT}}{\Delta} \tag{4} $$

where $\Delta = \frac{\sum_{j=1}^{J}(P_j - {P_j}^{GT}) \cdot \delta((P_j - {P_j}^{GT}) < TH)}{\sum_{j=1}^{J}\delta((P_j - {P_j}^{GT}) < TH)}$, $\delta(x<TH)=0$ and $\delta(x \geq TH)=1$, with $TH = \frac{\sum_{j=1}^{J}(P_j - {P_j}^{GT})}{J}$.

A. Offset Classifier

We collect the offset training data of twelve joints for the offset classifier training. After training, the offsets of similar poses are saved in the same leaf node. Based on the twelve body points, we compute the distances of the 3-D coordinates between the joints of the each body part and the joint of the head as split function. The spilt function f_θ is defined as

$$ f_\theta = (\|P_0 - P_a\|_2) - (\|P_0 - P_b\|_2) \tag{5} $$

where P is the body part joint, and parameter $\theta=\{a, b\}$.

For each set of split function parameters φ, we can divide the training data into two sub-groups. Here, we define the error function for the offset classifier training by computing the sum of all the offsets of the body part joints and dividing by the total number of the training sample in the nodes (the error average). Then, we subtract the offset of the body part joints in the node. The error function f_E is defined as

$$ f_E = \sqrt{\frac{\sum_{i=1}^{m}\left|F_i - \frac{\sum_{i=1}^{m} F_i}{m}\right|}{m}} \tag{6} $$

where m is the number of training data in the node, and $\boldsymbol{F}$ is the offset .

For similar poses, there are similar offsets in the same leaf node. These offsets need to be normalized due to they are object-dependent. Assuming n training offset samples in the

leaf node, we average the data in the node for the offset normalization of which the equation is defined as

$$Offset_t{}^j(I,\boldsymbol{P}) = \frac{\sum_{i=1}^{n}(\boldsymbol{F}i*\frac{(\sum_{i=1}^{n}\Delta_i)}{n})}{n}, \quad (7)$$

where $\boldsymbol{F} = \{F_j\}$ and $\boldsymbol{P} = \{P_j\}$ for j=1, 2,..12.

The offset classifier is a random forest. The offset training samples stored in the leaf node are used to compensate the estimated body part joints. In the leaf node, we have the average offset for compensating the pose. For pose $\boldsymbol{P}$, we have the offsets from the leaf node of the t^{th} decision tree, $\{Offset_t(I,\boldsymbol{P})\}$, and compute their mean as

$$Offset_{mean} = \frac{\sum_{t=1}^{T}\boldsymbol{Offset_t}(I,\boldsymbol{P})}{T} \quad (8)$$

For pose $\boldsymbol{P}$ and the t^{th} decision tree, we can obtain the corresponding offset in the leaf node as $Offset_t(I,\boldsymbol{P})$. Then, we add $offset_{mean}$ to the original location of the twelve joints of the body part. The compensation using the offset classifier is effective. For twelve body part joints, we have the 12×3-dimensional offset vectors stored in the leaf node.

B. Justification of Offset Compensation

To justify the compensated results, we examine the consistency between the current estimated pose and the real observation. After using body part classifier and offset classifier, we generate the skeleton of twelve joints. Incorrect offset compensation may occur and deteriorates the estimation results. Therefore, we need to verify the compensation process to improve the accuracy. We compare the compensated and the original results to validate the compensation by examining the depth map. The justification process is based on the matching between the silhouettes of the captured depth image and the body part joints as shown in Fig. 5.

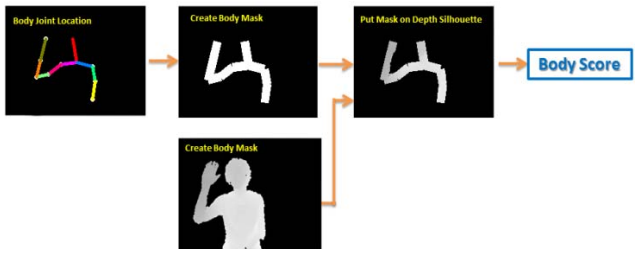


Fig. 5. Justification of offset compensation.

The justification process consists of the following steps: (1) Construct a body mask based on the estimated body part joints. (2) Put this mask on the depth silhouette image. (3) Compute the number of the pixels inside the mask silhouette. (4) Calculate the average depth of the mask of each body part. (5) Use the average depth as the weight. Let the set of body part as $P = \{P_i | i = 1,..12\}$, H_i=$\|P_i - P_{i-1}\|_2$, i=1…12 is the length of the i^{th} body part mask, and W is the width of the body part mask. The width is fixed as W=W_0. θ_i is the orientation of the i^{th} body part mask, and $\boldsymbol{I}$ is the binary silhouette. Justification process puts the weighted rectangle mask on each body part P_i with orientation θ_i. Here, we have a set of body part masks as $\boldsymbol{Mask} = \{Mask_1, \ldots, Mask_{12}\}$, and put these masks onto the depth map image to obtain the depth map for body part k as

$$BPD_k = \boldsymbol{I} * Mask_k \quad (9)$$

for k= *1….12*. Finally, we compute the average depth of each mask body part depth map as

$$BPDM_k = \frac{\sum_{y=1}^{m_k}\sum_{x=1}^{n_k} BPD_k(x,y)}{\sum_{y=1}^{m_k}\sum_{x=1}^{n_k}\delta(BPD_k(x,y)>0)} \quad (10)$$

for k= *1….12,* where n_k and m_k are the height and weight of the rectangle mask. The matching score is

$$\textbf{Score} = \prod_{k=1}^{12}(BPDM_k \cdot (\frac{\sum_{y=1}^{m_k}\sum_{x=1}^{n_k}\delta(BPD_k(x,y)>0)}{m_k*n_k}) \quad (11)$$

where BPD is the body part depth, and $BPDM_k$ is the mean of the k[th] body part depth.

VI. Experimental Results

In the experiment, Kinect is located with a distance 75~85 *cm* in front of the human operator. For action classifier training, we collect total depth images about 10,000 images (with resolution 640×480) from nine people. We generate three decision trees for Random Forest classifier, and select randomly 80% training data from the total depth image for decision tree training. Then we test the classifier and find the depth images that are miss-recognized and treated as training data for re-training. The parameters of the rectangle of the action classifier are: the length is l=140, the width is w=70, the shifting distance is s=60. The AdaBoost is constructed by about 1000 weak classifiers from 10000 training depth images.

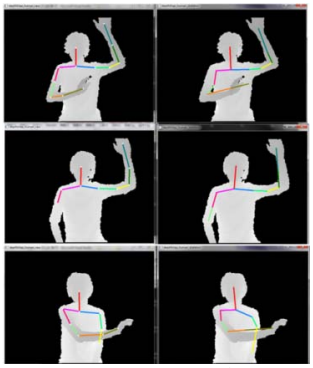
For the body part classifier, we collect nine peoples, 1000 depth images, and randomly sample 200 points in each image. Here, we select 200,000 sample points to train a decision tree. Each body part classifier consists of three decision trees. The $\boldsymbol{u}$ and $\boldsymbol{v}$ randomly generated with the limit of 100 pixels. To train the offset classifier, we collect the movement from 9 people from 1,000 depth images. We train three decision trees for the offset classifier. We randomly select the train sample to train the decision trees for 12 body part classifiers. The termination criteria for split node generation are the number of offsets is less than 3 or the value of the error function is 0.5.

Three depth videos with 900 depth images are tested. We manually label the action type and the joints of the body part in each depth image. We achieve the accuracy of 91.78% for the action type classification. Then the correction based on temporal correlation is applied which involves preceding and succeeding action types.

The length and the width of the rectangle feature, and the shifting distance will affect the recognition rate of the action classifier. Different rectangle parameters (length/width and shift distance s) will generate different accuracy rate. If we shift a smaller distance s, we can generate more overlapped rectangles and more features. The accuracy rate will decrease if we have too many features. It is because the average depth value of these rectangles is too similar. Considering too much detail will deteriorate the overall system performance. We may also move a larger distance s, and take little local features into account. The accuracy rate will also decrease. Then, we find the best aspect ratio of the rectangle that fits the shape of the arm.

For fixed rectangle area and shift distance s (=width×0.85), we change the aspect ratio of length/width for different accuracy rate. The accuracy rate is correlated with the area. With fixed aspect ratio and shift distance s, we may also change the area of the rectangle for different accuracy. If the area is small, little depth information is considered as the features, and large error occurs. If more depth area is used as features, the accuracy rate will also decrease. Here, we define the parameters of the rectangle as l=140, w=70, s=60.

With temporal correction, the accuracy rate of the recognized action type is 91.78%. To train the body part classifier, we choose 1000 depth images and manually label the joints in the depth images as the ground truth. The error is measured by the distance between the estimated body part joints and the ground truth. The average errors of all joints range from 5.8 pixels to 13.2 pixels. The smallest error occurs at left upper arm and the largest error is at left lower upper arm. Due to the hierarchical structure of human object, the error of the upper joints (*i.e.*, head, shoulders) is generally smaller than the error of lower joints (*i.e.*, elbows, palms, etc.). Fig. 6 shows the un-precise body part joints.



(a) (b)

Fig. 6. (a) The ground truth, (b) The result of the body part classifier.

To train the offset classifier, we manually label body part as the ground truth and compute the difference between the ground truth and the results of the body part classifier. Totally, we collect 1000 offset images for training. If the leaf node is not correctly classified, it may jeopardize offset compensation. So we need to justify the compensation.

To validate the offset compensation, we compare the result of compensation and uncompensated results based on the matching scores to determine which one is the more suitable. We do the justification based on the matching score of the estimated body part joints without/with offset compensation. We compute the matching scores to justify the effectiveness of offset compensation. Insufficient offsets training data stored in the leaf node cannot provide reliable joint correction statistics.

To avoid incorrect offset compensation, we propose the post-processing to justify the results of offset compensation. Fig. 7 shows that the final result of our method (in blue) is better than the original result (in red). Our offset classifier is better because we train the four different offset compensation classifiers using four different action training data sets to generate more coherent offset data stored in leaf nodes. Besides, we have applied different spilt functions in non-leaf nodes to create more discriminant decision tree for more precise offset compensation of the estimated joint locations

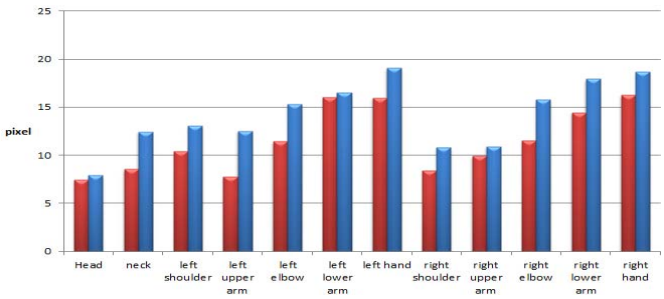


Fig. 7. The estimation error of the joint locations of two different offset classifiers. Our offset classifier in red and the original offsets classifier [3] in blue.

Since Microsoft Kinect SDK also provides human identification module, we compare the results of our method with the Kinect SDK of Microsoft as shown in Fig. 8. Our method illustrates better performance. The results of applying offset compensation are shown in Fig. 9.

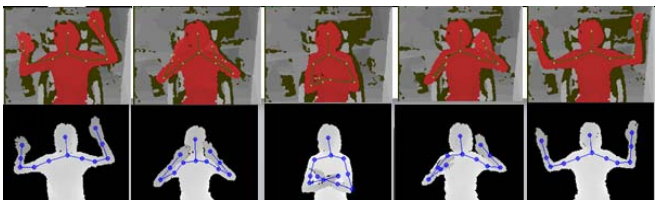


Fig. 8. Under the same environment, we compare the results of Microsoft SDK (red foreground) and ours (gray foreground). The error of our method is smaller than the Microsoft SDK.

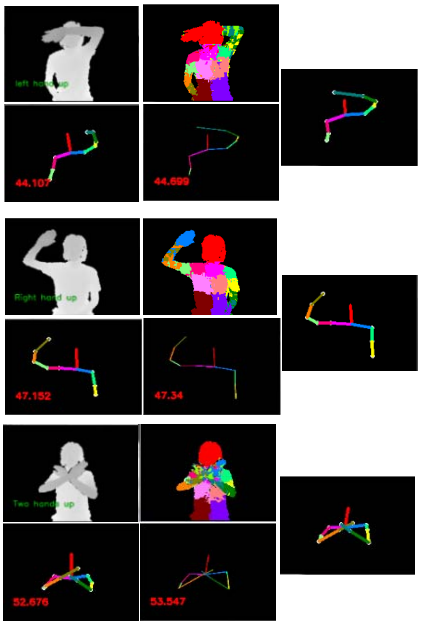
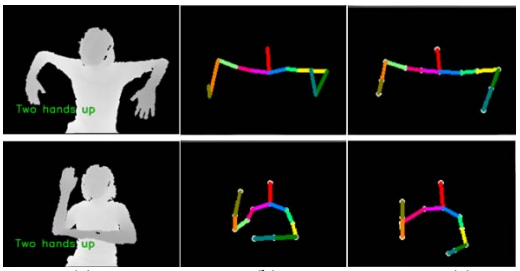


Fig. 9. The results of (a) the left hand raised, (b) the right hand raised, and (c) both hand raised. The upper left is the output of the action classifier, and the upper right is the result of the body part classifier. The lower left is the skeleton. The lower right is the result of the offset compensation. The right is the final output. The red digits indicate the matching score.

For novel input pose, the estimation results may not be correct. For each input pose with similar poses in the training

data set, it may generate correct results. We may estimate the joint locations of some poses which are not in the training set as shown in Fig. 10. If the input pose is very different from the training poses, it will not generate the correct result as shown in Fig. 10(b). Because of the complexity of human poses, we may not find the similar pose in the training set. To classify the pose with self-occlusion, we need include this pose in the training data. The results are shown in Fig. 10(c). Finally, we apply our method to estimate the motion parameters of some novel human poses. The results are shown in Fig. 11.



(a) (b) (c)

Fig. 10. (a) Input pose, (b) The results of the classifier trained without this pose. (c) The results of the classifier trained with this pose.

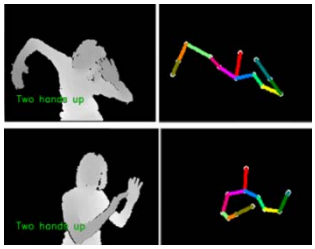


Fig. 11. The correct estimation results for complete novel poses.

VII. CONCLUSION

In this paper, we have proposed motion capturing system for human upper body motion. It consists of hybrid action type classifier, body part classifiers, offset compensation, and justification. The new concept is to compensate the result of body classifier by using the offset classifier and justifying the compensation for a better performance of estimating the body part joints.

REFERENCES

[1] J. Shotton, A. Fitzgibbon, M. Cook, T. Sharp, M. Finocchio, R. Moore, A. Kipman, and A. Blake, "Real-Time Human Pose Recognition in Parts from Single Depth Images," IEEE CVPR, 2011.

[2] V. Ganapathi, C. Plagemann, D. Koller, and S. Thrun,"Real time motion capture using a single time-of-flight camera," IEEE CVPR 2010.

[3] W. Shen, K. Deng, and X. Bai, "Exemplar-Based Human Action Pose Correction and Tagging," IEEE CVPR 2012.

[4] A. Baak, M. Müller, G. Bharaj, H.-P. Seidel, and H. Theobalt, "A Data-Driven Approach For Real-Time Full Body Pose Construction From A Depth Camera," IEEE ICCV 2011.

[5] S. Wang, H. Ai, and T. Yamashita, "Top-Down /Bottom-Up Human Articulated Pose Estimation Using AdaBoost Learning," IEEE ICPR 2010.

[6] T. Moeslund, A. Hilton, and V. Kr¨uger. "A Survey Of Advances In Vision-Based Human Motion Capture And Analysis," CVIU 2006.

[7] R. Poppe, "Vision-based human motion analysis: An overview," CVIU 2007.

[8] Microsoft Corp. Redmond WA. Kinect for Xbox 360.

[9] L. A. Schwarz, A. Mkhitaryan, "Estimating Human 3D Pose from Time-of-Flight Images Based on Geodesic Distances and Optical Flow," AFGR, 2011

[10] S. Sempena and N. U. Maulidevi, "Human Action Recognition Using Dynamic Time Warping," ICEE 2011.

[11] R. Yang , L. Ren, and M. Pollefeys, "Accurate 3D Pose Estimation From a Single Depth Image." ICCV,2011.

[12] J. Charles and M. Everingham, "Learning shape models for monocular human pose estimation from the Microsoft Xbox Kinect." ICCV,2011.

[13] C. Plagemann, V. Ganapathi, D. Koller, and S. Thrun. "Real-time identification and localization of body parts from depth images." ICRA 2010.

[14] L. Breiman, Random forests. Mach. Learning, 45(1):5–32, 2001.

[15] G. Fanelli, J. Gall, L. Van Gool, "Real Time Head Pose Estimation with Random Regression Forests," ICPR 2010.

[16] R.Wang and J. Popovi´c, "Real-Time Hand-Tracking with A Color Glove," ACM SIGGRAPH 2009.

[17] C. Bregler and J. Malik, "Tracking People with Twists And Exponential Maps," IEEE CVPR 1998.

[18] H. Yang and S. Lee. "Reconstructing 3D Human Body Pose from Stereo Image Sequences Using Hierarchical Human Body Model Learning," IEEE ICPR 2006.

[19] L. Sigal, S. Bhatia, S. Roth, M. Black, and M. Isard, "Tracking Loose Limbed People," IEEE CVPR 2004.

[20] T. Sharp. "Implementing decision trees and forests on a GPU." ECCV 2008.

[21] Ho, T. Kam, "Random Decision Forest," The 3rd ICDAR Montreal, QC, Aug. 1995.

[22] Y. Freund and R. E. Schapire, "A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting," Journal of Computer and System Sciences, 1996

[23] N. H. Lehment, M. Kaiser, and G. Rigol, "Using Segmented 3D Point Clouds for Accurate Likelihood Approximation in Human Pose Tracking," ICCV, 2011.

[24] L. Breiman, Bagging Predictors, Machine Learning,p123-p140 ,1996

[25] T. K.Ho, "The Random Subspace Method for Constructing Decision Forests," IEEE Trans. on PAMI, 1998.

[26] J.R. Shaw, QuickFill: An efficient flood fill algorithm. Online article: http://www.codeproject.com/gdi/QuickFill.asp,2005.

[27] R. Adams and L. Bischof, " Seeded Region Growing," *IEEE Trans. on PAMI* , vol. 16, no.6, 1994.